
TP noté - durée : 2 heures - documents autorisés : transparents de cours

ATTENTION :

- Les seuls documents autorisés sont les transparents du cours couvrant la matière vue jusqu'ici.
 - Les seules notions permises sont celles vues jusqu'ici, ce qui exclut entre autres `set`, `dict`, les slices et les compréhensions de liste.
 - Les fonctions demandées **ne peuvent pas modifier** les listes qu'on leur fournit.
 - Les doctests fournis sont une indication de ce que vous devez faire. Si un test ne passe pas, c'est que le code que vous avez écrit ne répond pas à la question. Le fait que tous les doctests passent ne garantit pas, en revanche, que votre code est correct.
 - Les questions sont indépendantes, vous pouvez y répondre dans l'ordre que vous souhaitez.
 - Vous avez le droit de ré-utiliser des fonctions écrites pour d'autres questions, et d'écrire en plus des fonctions qui ne sont pas demandées dans l'énoncé.
 - Il ne faut pas modifier les noms des questions ni les doctests.
-

Vous devez impérativement répondre dans le fichier `tpnote2025.py` fourni. Vous ne pouvez pas modifier les noms des fonctions demandées ni les doctests associés.

Exercice 1 (Traitement de texte)

- (a) Écrivez une fonction `positions_lettre(chaine, lettre)` renvoyant la liste de toutes les positions où `lettre` apparaît dans `chaine`. Par exemple :

```
>>> positions_lettre("programmation", "a")
[5, 8]
```

- (b) Écrivez une fonction `nombre_caracteres_differeents(chaine)` renvoyant le nombre de caractères différents de `chaine`. Par exemple :

```
>>> nombre_caracteres_differeents("programmation")
9
```

- (c) Écrivez une fonction `mot_le_plus_long(phrase)` renvoyant le mot le plus long d'une chaîne de caractères donnée. En cas d'ambiguïté, renvoyez le premier de ces mots. Par exemple, (ici "longue" est aussi de longueur maximale)

```
>>> mot_le_plus_long("Ceci est une phrase qui n'est pas très longue")
'phrase'
```

Exercice 2 (Séquences)

Un biologiste a besoin de manipuler des séquences d'ADN. Écrivez les fonctions de base nécessaires à son travail.

- (a) Écrivez une fonction `est_adn(chaine)` vérifiant qu'une chaîne de caractères donnée est bien une séquence d'ADN. Cela n'est vrai que si elle ne contient que des `'A'`, `'C'`, `'G'` et `'T'`. Vous pouvez supposer que `chaine` est en majuscules.
- (b) **Sans utiliser `str.count`**, écrivez une fonction `composition(chaine)` affichant la proportion de chaque lettre dans la séquence donnée. Par exemple :

```
>>> composition("CAACATCACAAGAAGT")
50.0 % de A
25.0 % de C
12.5 % de G
12.5 % de T
```

- (c) Écrivez une fonction `complement(chaine, motif)`, qui renvoie le complément d'une séquence de nucléotides, c'est-à-dire la séquence lue à l'envers et dans laquelle chaque 'A' est remplacé par un 'T' (et inversement) et chaque 'C' est remplacé par un 'G' (et inversement). Par exemple :

```
>>> complement("GACT")
"AGTC" # GACT à l'envers = TCAG, qui devient AGTC après remplacements
```

Exercice 3 (ISBN)

Une chaîne ISBN identifie de manière unique un livre à l'aide de 10 caractères — par exemple : "0262367505". Dans ce format, les deux premiers chiffres désignent une langue, les quatre suivants un éditeur, et les trois suivants un titre ; le dernier chiffre sert de contrôle, et est le seul caractère qui peut valoir 'X', interprété comme la valeur 10.

- (a) Écrivez une fonction `isbn_vers_donnees(isbn)`, qui renvoie sous la forme d'une liste de trois codes les informations décrites ci-dessus au départ d'une chaîne `isbn` valide. Par exemple :

```
>>> isbn_vers_donnees("0262367505")
["02", "6236", "750"]
```

- (b) Toutes les chaînes de 10 chiffres ne sont pas forcément des ISBN valides. Écrivez une fonction `est_isbn_correct(chaine)`, qui renvoie `True` si `chaine` est une chaîne ISBN correcte et `False` sinon. La chaîne est correcte si et seulement si la somme $10 * \text{chiffre}_1 + 9 * \text{chiffre}_2 + \dots + 2 * \text{chiffre}_9 + \text{chiffre}_{10}$ est divisible par 11. Par exemple :

```
>>> est_isbn_correct("0262367505")
True
>>> est_isbn_correct("038518915X")
True
>>> est_isbn_correct("3141592653")
False
```

Exercice 4 (Bourse)

Un investisseur désire écrire un programme pour se faire conseiller dans ses investissements. Aidez-le en écrivant les fonctions suivantes.

- (a) Pour pouvoir déterminer si une action est en train de monter et depuis combien de temps, écrivez une fonction `plus_long_suffixe_croissant(valeurs)`, qui renvoie la longueur du plus long suffixe croissant (non strictement) d'une liste de valeurs. Par exemple :

```
>>> plus_long_suffixe_croissant([3, 2, 1, 2, 2, 3])
4
```

- (b) L'investisseur souhaite savoir si une action vaut la peine d'être achetée. Écrivez une fonction `achat_interessant(valeurs)`, qui renvoie une liste de trois éléments : `True` si la valeur actuelle (la dernière entrée de la liste `valeurs`) est strictement inférieure à la moyenne des autres valeurs, `False` sinon, suivi de la valeur actuelle et de la moyenne calculé. Par exemple :

```
>>> achat_interessant([3, 2, 4, 2, 8, 5, 6])
[False, 6, 4.0]
>>> achat_interessant([10, 9, 8, 7, 6, 5])
[True, 5, 8.0]
```

- (c) L'investisseur désire savoir, pour une action donnée, quels ont été les plus gros gains et les plus grosses pertes. Écrivez une fonction `gains_et_pertes_max(valeurs)`, qui

renvoie un couple de deux pourcentages : le plus gros gain suivi de la plus grosse perte (si l'une de ces deux valeurs n'existe pas, remplacez-la par 0). Par exemple :

```
>>> gains_et_pertes_max([4, 1, 4, 2, 8, 5, 6])
[700.0, -75.0]
# gain max:  passage de 1 à 8, l'action a gagné 700 %
# perte max: passage de 4 à 1, l'action a perdu  75 %
```

Vous pouvez supposer que **valeurs** ne contient jamais 0.